

Modeling Speculative Decoding

Mihika Dusad
mdusad@mit.edu

Jason Ng
jing06@mit.edu

I. KEY INSIGHTS AND INNOVATIONS

- **Speculative decoding should be analyzed as a hardware-mapping problem, not just an algorithmic one.** While prior work emphasizes end-to-end speedup, our experiments show that the realized benefit depends strongly on how speculative verification is mapped onto accelerator resources. In particular, per-token latency and energy are governed not only by fewer serial target-model decode steps, but also by memory traffic, reuse, and the placement of K/V accesses in the memory hierarchy.
- **The best lookahead is moderate rather than maximal.** Across our sweeps, speculative decoding improves over baseline decode, but the benefit is non-monotonic in lookahead γ . Small γ underutilizes available parallelism, while large γ increases verification overhead and wasted speculative work when acceptance is imperfect. Our gamma sweep shows that $\gamma = 4$ is the most consistently favorable operating point in the tested regime, highlighting the need for hardware-aware tuning of speculative decode parameters.
- **Hardware-specialized support for speculative verification matters more than simple buffer removal.** Removing the global buffer alone has little effect on latency, whereas fused QKV execution provides a much larger gain by reducing repeated projection overhead. Furthermore, our speculative-decoding-aware `arch_kv_buffer.yaml` improves both latency and energy across all shared operating points by introducing a dedicated KV-cache buffer and a verification-oriented spatial fanout. This shows that speculative decoding opens up concrete architectural opportunities in projection fusion and KV-cache movement.

II. CONTRIBUTIONS

- **Mihika Dusad:** Led experiment design, parameter sweeps, and analysis of how context length, lookahead γ , and token acceptance probability α affect speculative decoding. Also contributed to interpreting the hardware trends, preparing plots, and writing the report.
- **Jason Ng:** Led workload mapping, accelerator-oriented modeling, and architectural analysis in AccelForge. Designed and evaluated the fused-QKV and KV-buffer architectural variants, analyzed mapping behavior, and contributed to the final write-up and experimental interpretation.

III. ARTIFACT

- **GitHub repository:** <https://github.com/JasxnNg/65930-final>
- **Final report:** Included with submission.
- **Poster:** Included with submission.
- **Additional materials:** Experiment CSVs, figure-generation notebooks, workload YAMLS, and architecture YAMLS are available in the repository.

Modeling Speculative Decoding

Mihika Dusad
mdusad@mit.edu

Jason Ng
jing06@mit.edu

Abstract—Speculative decoding is a promising technique for accelerating large language model inference, but its reported gains are usually analyzed at the algorithmic or system level rather than from the perspective of accelerator behavior. In this work, we model speculative decoding directly in AccelForge on a TPU-v4-inspired architecture and evaluate its cost based on per-token latency and per-token energy. We sweep over context length, lookahead γ , and token acceptance probability α to study when speculative decoding is beneficial and which hardware-facing effects explain the gain. Our results show that speculative decoding consistently outperforms standard KV-cached autoregressive decoding in the tested regime, with benefits that grow with context length but vary non-monotonically with lookahead. We further show that removing the global buffer has little effect on latency, while fusing QKV produces the largest immediate architectural win, reducing speculative per-token latency by about 25% and energy by about 16% relative to the no-global-buffer design. Finally, a speculative-decoding-aware KV-buffer architecture lowers latency and energy by about 20% and 17%, respectively, on the shared sweep. These results suggest that the speedup from speculative decoding comes not only from fewer serial target-model decode steps, but also from hardware-aware optimizations in projection fusion and KV-cache movement.

I. INTRODUCTION

The Transformer architecture [1] has become an increasingly popular framework for many modern deep learning applications, including large language models (LLMs), enabling strong performance gains across tasks like reasoning, generation, and multi-step decision-making. However, these gains do come at a significant cost, and inference for large models is both computationally expensive and energy-intensive, creating bottlenecks in latency, throughput, and deployment scalability. Most of these transformer-based LLMs consist of billions of parameters, making it difficult to deploy them directly onto consumer hardware. Inference-time optimizations are an important consideration, especially as models grow bulkier and more complex. Yet, as highlighted in prior work, even optimized models remain difficult to serve efficiently due to their size and memory demands [2].

LLM inference can be divided into two phases: prefill and decode. While prefill is largely compute-bound, decode is dominated by memory bandwidth due to repeated memory accesses of model weights and the tensors in the KV cache.

As a result, many recent works focus on optimizing decode through techniques such as KV caching [5], paged attention [6], and the main focus of our paper, speculative decoding [7]. Speculative decoding has emerged as a promising method for accelerating inference. It works by employing a smaller draft model to stream tokens, and then verifying these generated tokens using the larger target model.

Despite reported 2–3 $\times$ speedups, it remains unclear why or how speculative decoding achieves these gains from a hardware perspective. Existing work primarily evaluates speculative decoding at the algorithmic or software level, focusing on throughput or latency improvements, without fully characterizing how these gains map onto specific hardware behavior. In particular, key questions remain about how factors such as lookahead length (γ), context length, and token acceptance probability (α) influence memory traffic, computational intensity, and overall energy efficiency.

This paper studies speculative decoding from a hardware-mapping perspective. Rather than only measure aggregate speedup, we model speculative decoding workloads on a TPU-like baseline architecture and evaluate them in AccelForge using per-token latency and per-token energy metrics. Our first contribution is a baseline characterization of speculative decoding on the original `arch.yaml`, compared against standard KV-cached autoregressive decoding. Our second contribution is a scaling study over context length, lookahead γ , and token acceptance probability α , which reveals how verification overhead, reuse, and wasted speculative work interact. Our third contribution is connecting workload structure to direct hardware-facing optimizations by evaluating fused QKV execution, showing that changes to how attention projections are mapped can produce much larger latency gains than characterization alone. Together, these experiments help explain not only how speculative decoding is beneficial, but where its benefits come from and when they begin to diminish.

II. RELATED WORKS

Prior work on LLM inference optimization can be broadly divided into optimizations addressing model requests (the system runtime itself), and decode-level optimizations. On the systems side, PagedAttention improves memory efficiency by virtualizing KV-cache allocation and reducing fragmentation during generation [6]. KIVI reduces KV-cache footprint through low-bit asymmetric quantization [5]. DistServe separates prefill and decode across different hardware resources to better match their distinct bottlenecks, while Sarathi-Serve uses chunked prefill to improve throughput-latency tradeoffs

during serving [3], [4]. These frameworks are highly effective at improving throughput, TTFT, and memory utilization, but they primarily study system behavior rather than how speculative workloads map onto accelerator structures.

A separate line of work addresses the inherent sequential nature of autoregressive decoding. Leviathan et al. introduced speculative decoding, in which a draft model generates candidate tokens and a target model verifies them in parallel without changing the output distribution [7]. Medusa extends this idea by attaching multiple decoding heads to a single backbone model and verifying a tree of candidate continuations [8]. These methods demonstrate that reducing the number of serial target-model invocations can substantially speed up decoding, but they mostly evaluate the gain at the algorithmic or end-to-end system level.

Our work is closest to this second category, but we study speculative decoding as a hardware workload rather than only as a decoding algorithm. Specifically, we ask how speculative verification, lookahead, and acceptance probability affect computational intensity, memory traffic, and loop mappings, and we evaluate whether hardware-facing optimizations such as fused QKV execution can better exploit that structure. In this sense, our work complements prior speculative decoding research: instead of treating accelerator behavior as a black box, we use mapping-based analysis to identify which components of the speculative pipeline are responsible for energy savings or latency speedup.

III. IDENTIFYING THE PROBLEM

The problem we seek to address is that it is unclear exactly which components of speculative decoding contribute towards the achieved 2-3x speedup in inference, specifically which parts of the workload actually produce the observed speedup once mapped onto an accelerator. We therefore model speculative decoding directly in AccelForge [10] and evaluate it on a TPU-v4 [11] inspired baseline architecture defined in `arch.yaml`. We represent the draft and verifier workloads using the GPT-style KV-cache YAMLS in our codebase. We gather per-token latency and per-token energy results for each mapped workload, which allows us to compare different lookahead values, context lengths, and acceptance probabilities.

Beyond baseline characterization, our project also studies direct hardware-facing optimizations. In particular, we evaluate a fused-QKV variant that executes the attention projections together rather than as separate operations, reducing repeated projection overhead and improving reuse of shared inputs. This allows us to move from simply characterizing speculative decoding to identifying concrete mapping and architectural choices that better support it.

IV. EXPERIMENTAL SETUP

We ran our experiments in AccelForge to test our hypotheses and to gain a deeper understanding of the intricacies of the relationships between lookahead, context length, and per-token energy and latency. Our experiments used a TPU-v4-inspired architecture model because our workloads are

dominated by large and dense matrix multiplications, i.e. little opportunity for reuse exists here. Concretely, the architecture in our `arch.yaml` models off-chip main memory, a shared global buffer, a local buffer, scalar operations, and a PE-level MAC hierarchy with explicit reuse dimensions.

In our experiments, *context length* refers to the number of previously processed tokens already present in the sequence and, equivalently, the amount of prior KV-cache tensors that the verifier must attend over. *Lookahead*, denoted by γ , is the number of draft tokens proposed before running one target-model verification step; in our setup, the terms *lookahead* and γ are interchangeable. Finally, α denotes the token acceptance probability, i.e., the probability that a drafted token is accepted by the target model during the verification stage. These three parameters jointly determine the tradeoff between speculative parallelism, verification overhead, and wasted work from rejected branches.

For each configuration, we compared two execution modes. The baseline mode was a standard KV-cached autoregressive decoding, implemented as γ serial target-model decode steps. The speculative mode executed γ draft-model decode steps followed by one target-model verification step. We reported all results as per-token metrics so that configurations with different lookahead values would remain comparable. To account for imperfect draft acceptance, we normalized speculative cost by the expected number of accepted tokens, parameterized by acceptance probability α . This allowed for the experiments to capture both the benefit of fewer target-model invocations and the penalty of wasted work when speculative branches are rejected.

We ran three separate experiments. In Experiment 1, we simply aimed to prove that our modeling of speculative decoding worked correctly, yielding the expected latency speedup of 2-3x. It validated the baseline speculative-decoding model on the unmodified TPU-v4-like `arch.yaml`, sweeping context lengths of 128 to 1024, lookahead γ of 1 to 10, and acceptance probability α from 0.4 to 0.95. Experiment 2 studied scaling with context length; we plotted per-token energy and per-token latency versus context length for multiple fixed lookahead values in order to determine whether longer contexts better amortized verification overhead. Lastly, Experiment 3 evaluated a fused-QKV variant in which the three attention projections were executed together rather than as separate projection stages.

In order to actually examine the workload and hardware, we needed to first construct a mapping onto the respective hardware. AccelForge allows us to both create these mappings relatively quickly [13]. Furthermore, we examined the mapping with the loops fused, implicitly modeling speculative decoding with flash attention as we fuse the softmax [12].

V. RESULTS AND DISCUSSION

At a high level, our results show that speculative decoding improves per-token latency over standard autoregressive decoding across the operating points we study, and, at the plotted acceptance probabilities, it also reduces per-token energy.

However, the gains are not explained by speculation alone. Our experiments show that the realized benefit depends strongly on how the speculative workload is mapped to hardware. In particular, larger context lengths amplify the baseline cost of decode, moderate lookahead values provide the best trade-off, and architectural changes that better match speculative attention—especially fused QKV execution and a KV-aware memory hierarchy—produce the largest improvements.

These results lead to three main conclusions. First, speculative decoding becomes more useful as context length grows because the baseline decode cost increases rapidly with sequence length. Second, not all hardware changes matter equally: removing the global buffer has very little effect on per-token latency, while fusing the QKV substantially lowers both latency and energy. Third, an architecture designed more directly around speculative verification, namely our KV-buffer design, consistently improves both metrics across all shared points in the sweep. Together, these findings suggest that the source of speculative decoding’s speedup is not just fewer target-model decode steps, but a combination of reduced serial work, better reuse, and more favorable data movement patterns.

A. Speculative decoding scales better with context

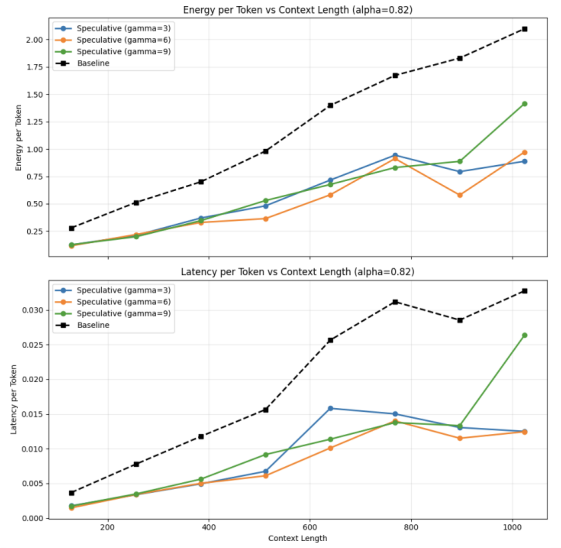


Fig. 1. Per-token latency and energy versus context length for baseline and speculative decoding at fixed acceptance probability ($\alpha = 0.82$).

Figure 1 shows per-token energy and per-token latency versus context length for baseline decode and speculative decoding with $\gamma \in \{3, 6, 9\}$ at fixed $\alpha = 0.82$. The first clear trend is that baseline decode becomes steadily more expensive as context length increases. Both the energy and latency curves for the baseline rise sharply with context length, which is consistent with the growing cost of repeatedly reading the KV cache and re-running the target model one token at a time.

In contrast, all three speculative configurations remain below the baseline in both plots. This confirms that our

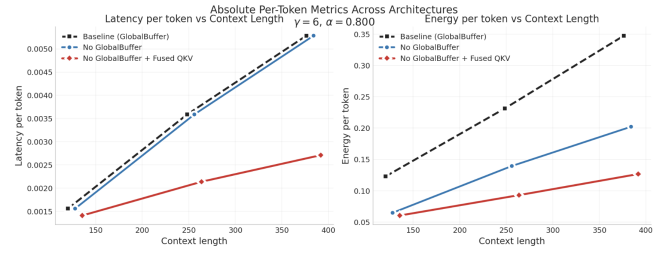


Fig. 2. Architectural ablation of speculative decoding: baseline, no-global-buffer, and fused-QKV designs.

speculative-decoding model is capturing the expected throughput benefit: amortizing one target-model verification step across multiple drafted tokens lowers per-token cost. At the same time, the speculative curves are not strictly monotonic in γ . A moderate lookahead, especially $\gamma = 6$, performs best or near-best across much of the sweep, while $\gamma = 9$ becomes less stable at larger context lengths. This is an important result: increasing lookahead does not automatically translate into better hardware efficiency. Larger γ values expose more parallelism, but they also increase verification work and magnify wasted speculative computation when acceptance is imperfect. The main insight from Figure 1 is therefore that speculative decoding has a context-dependent sweet spot rather than a universally optimal lookahead.

B. Global-buffer removal against fused QKV

Figure 2 compares three architectures at fixed $\gamma = 6$ and $\alpha = 0.80$: the original `arch.yaml` baseline with the global buffer, a version with the global buffer removed, and a third version that removes the global buffer and additionally fuses QKV. This experiment isolates which architectural changes meaningfully improve speculative decoding.

The first result is that removing the global buffer alone has very little effect on latency. The baseline and no-global-buffer curves nearly overlap in the latency plot, indicating that the global buffer is not the primary bottleneck on the critical path for this workload. The energy plot shows a somewhat larger gap, meaning that buffer removal changes where data movement energy is spent even if it does not substantially improve time per token. By far the largest improvement comes from fused QKV. The fused-QKV curve lies well below the other two in both latency and energy across all plotted context lengths, showing that combining the three attention projections is a much more effective optimization than simply deleting one buffer level. Since softmax was fused in all of these configurations, the difference in Figure 2 can be attributed to the global-buffer ablation and QKV fusion itself, rather than to a change in softmax handling. The key takeaway is that projection-level fusion is the dominant architectural win in our baseline design.

C. KV-buffer architecture improves speculative verification

Figure 3 evaluates our speculative-decoding-aware `arch_kv_buffer.yaml` against the original `arch.yaml`

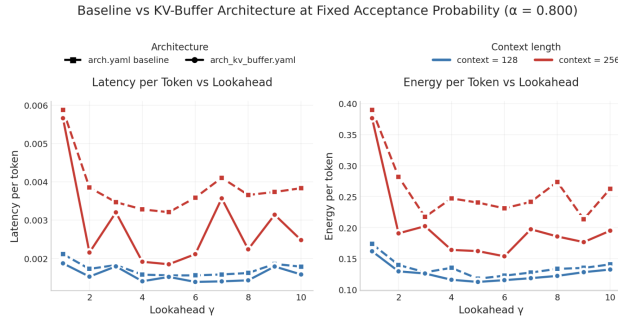


Fig. 3. Per-token latency and energy versus lookahead for the baseline and KV-buffer architectures.

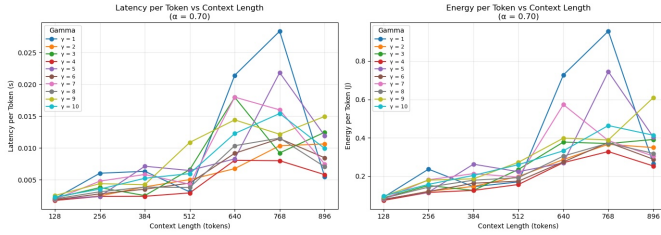


Fig. 4. Per-token latency and energy across context lengths for a sweep of lookahead values at fixed acceptance probability ($\alpha = 0.70$).

baseline at fixed $\alpha = 0.80$, while sweeping lookahead γ for context lengths 128 and 256. The key difference between these two architectures is that the baseline `arch.yaml` uses a generic shared `GlobalBuffer` for all tensors, whereas `arch_kv_buffer.yaml` introduces a dedicated `KVCacheBuffer` that stores only the K and V tensors. In other words, the speculative-decoding-aware variant explicitly separates KV-cache traffic from the rest of the activation and intermediate traffic. It also adds a `verify_m` spatial fanout in the processing-element array, allowing multiple speculative positions to be verified in parallel while still reusing weights. Together, these changes make the modified architecture more directly aligned with the access pattern of speculative verification, which repeatedly rereads prior K/V state and only appends a small amount of new data each round.

Unlike the global-buffer ablation, this experiment targets the repeated K/V accesses that dominate speculative verification. The resulting curves show that the KV-buffer design consistently lowers both latency per token and energy per token across the entire shared sweep. This improvement is not just visual; it is consistent across every shared operating point. Comparing the two CSV sweeps directly, the KV-buffer architecture improves speculative latency and speculative energy in all 418 common configurations. Averaged across those points, it reduces latency per token by about 20% and energy per token by about 17%. The benefit is also larger at context length 256 than at 128, which supports the architectural intuition behind the design: as context grows, verification rereads more K/V state, so a dedicated KV-oriented memory path becomes increasingly useful. The overall shape of the curves remains

non-monotonic in γ , meaning the architecture does not eliminate the algorithmic tradeoff in lookahead, but it shifts the entire operating region downward. In other words, the KV-buffer architecture does not change the fact that moderate γ values are best; it makes speculative decoding uniformly cheaper within that regime.

To make the lookahead tradeoff more explicit, Figure 4 sweeps γ from 1 to 10 across increasing context lengths at fixed $\alpha = 0.70$. We included this figure because the KV-buffer experiment alone shows that a speculative-decoding-aware architecture helps, but it does not directly identify which lookahead value is best once verification overhead and imperfect token acceptance are both taken into account. Figure 4 therefore isolates the effect of lookahead itself and helps identify the most favorable speculative operating point.

The key observation is that $\gamma = 4$ is the most consistently optimal lookahead in this experiment, producing the lowest or near-lowest per-token latency and per-token energy across the plotted context lengths. This supports the broader trend seen in our other experiments: the best speculative-decoding configuration comes from balancing parallelism against verification overhead rather than simply maximizing lookahead. The rationale for this behavior is intuitive. When γ is too small, speculative decoding does not fully amortize the cost of the target-model verification step, so the hardware is underutilized and the latency benefit is limited. When γ becomes too large, the verifier must process a larger speculative branch and more drafted work is exposed to rejection, increasing both wasted computation and memory traffic. A moderate lookahead such as $\gamma = 4$ therefore provides the best compromise, exposing enough parallel work to reduce serial decode cost without incurring the larger verification and rollback penalties seen at higher γ . Figure 4 is important because it shows that the optimal speculative-decoding regime is not monotonic in lookahead, and that hardware-aware tuning of γ is necessary to achieve the best latency and energy efficiency.

Overall, our results show that speculative decoding should be understood as both an algorithmic and architectural optimization. Baseline speculative decoding already improves latency relative to autoregressive decode, especially at larger context lengths, but the biggest hardware gains come from aligning the accelerator with the structure of the workload. Fused QKV reduces repeated projection overhead, while the KV-buffer architecture directly improves the verifier’s repeated K/V accesses. This is the central conclusion of the paper: the practical benefit of speculative decoding is not only that it reduces the number of serial target-model steps, but that it creates new opportunities for hardware specialization in projection fusion and KV-cache movement.

Our final repository can be found at this GitHub repo.

VI. CONCLUSIONS AND FUTURE WORK

This work shows that speculative decoding should be viewed as both an algorithmic optimization and a hardware-mapping problem. On our TPU-v4-inspired baseline architecture, speculative decoding reduces per-token latency relative

to standard KV-cached autoregressive decoding, and its advantage generally grows with context length as baseline decode becomes increasingly expensive. At the same time, the benefit is not monotonic in lookahead: moderate values of γ perform best because they balance parallel verification against the extra work introduced by longer speculative branches and imperfect acceptance.

Our architectural experiments further clarify which hardware changes matter. Removing the global buffer alone has little effect on latency, indicating that it is not the dominant critical-path bottleneck for this workload. In contrast, fused QKV execution provides a much stronger improvement by reducing repeated projection overhead and improving reuse of shared inputs, while the KV-buffer architecture more directly targets the repeated K/V accesses that dominate speculative verification. Together, these results show that the realized speedup of speculative decoding is driven not only by fewer target-model decode steps, but also by better dataflow and memory-hierarchy alignment.

There are several useful directions for future work. First, our current model uses an expected-value acceptance formulation rather than explicitly modeling rollback and branch-state management, so a more detailed control-flow model could better capture the cost of rejected speculative tokens. Second, we evaluate only a TPU-like accelerator and a small set of architectural changes; extending the study to other memory hierarchies, precision formats, and serving settings would help determine how general these conclusions are. Finally, techniques such as Medusa-style multi-head drafting, more detailed KV-cache layouts, and area-aware hardware comparisons could further connect speculative decoding algorithms to practical accelerator co-design.

VII. LLM USAGE

- We used an LLM to help us mainly in the initial literature review phase of the project. The LLM was very helpful in generating ideas for research, indicating gaps in current work, and finding similar papers. We also used an LLM to help us create some of our plots and figures from the data that we had collected from our experiments.
- The LLM was most helpful for the literature review (related works section). Finding similar papers would have been far more tedious otherwise.
- The LLM was not helpful in creating mappings files and writing code to generate mappings. We found that it lacks a general intuition for hardware components and AccelForge documentation.

REFERENCES

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. Advances in neural information processing systems, 30.
- [2] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.
- [4] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.
- [5] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," in *Proceedings of the 41st International Conference on Machine Learning*, PMLR, vol. 235, 2024, pp. 32332–32344.
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.
- [7] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proceedings of the 40th International Conference on Machine Learning*, PMLR, vol. 202, 2023, pp. 19274–19286.
- [8] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," arXiv preprint arXiv:2401.10774, 2024.
- [9] R. Parthasarathy and R. Shuttleworth, "Analyzing Inference Optimizations for Transformers," MIT 6.5930 Final Project Report, 2025.
- [10] T. Andrulis and M. Gilbert, *AccelForge*, software, MIT License. [Online]. Available: <https://github.com/Accelergy-Project/acclforge>
- [11] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23)*, Orlando, FL, USA, June 2023, pp. 1–14, doi: 10.1145/3579371.3589350.
- [12] T. Andrulis, M. Gilbert, V. Sze, and J. S. Emer, "Fast and Fusiest: An Optimal Fusion-Aware Mapper for Accelerator Modeling and Evaluation," *arXiv preprint arXiv:2602.15166*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.15166>
- [13] M. Gilbert, T. Andrulis, V. Sze, and J. S. Emer, "The Turbo-Charged Mapper: Fast and Optimal Mapping for Accelerator Modeling and Evaluation," *arXiv preprint arXiv:2602.15172*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.15172>